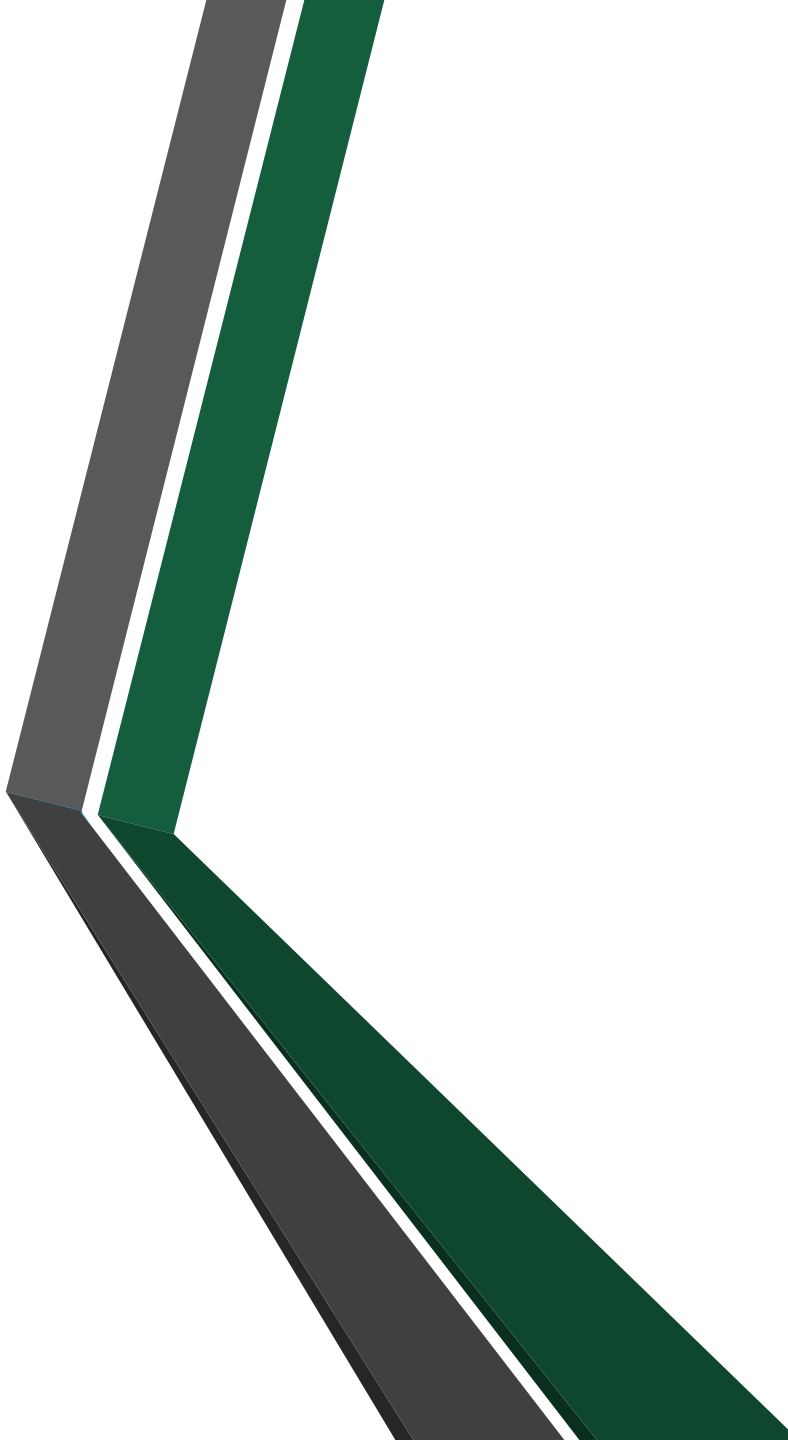




BAZE DE DATE

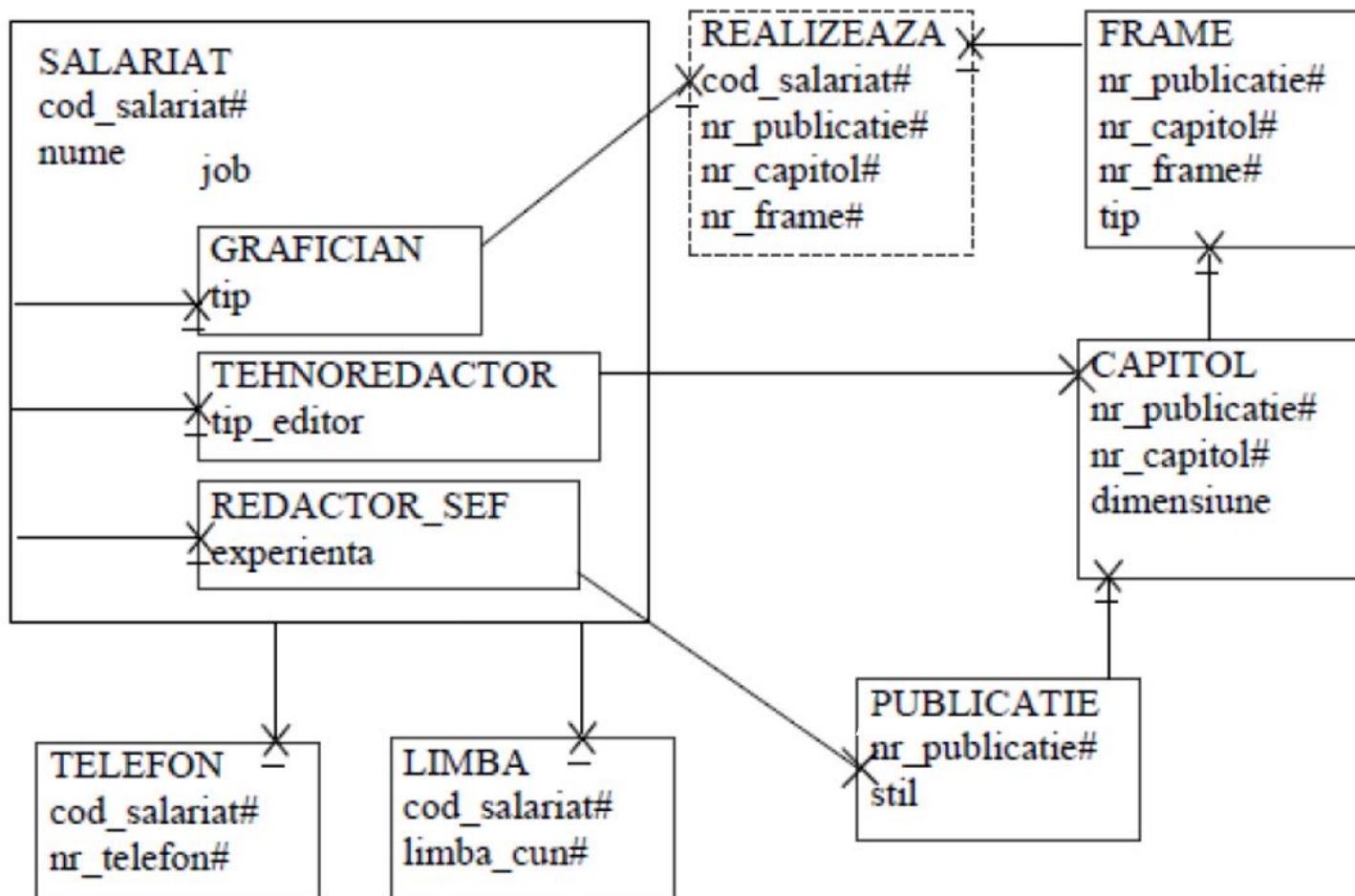
CURS 9

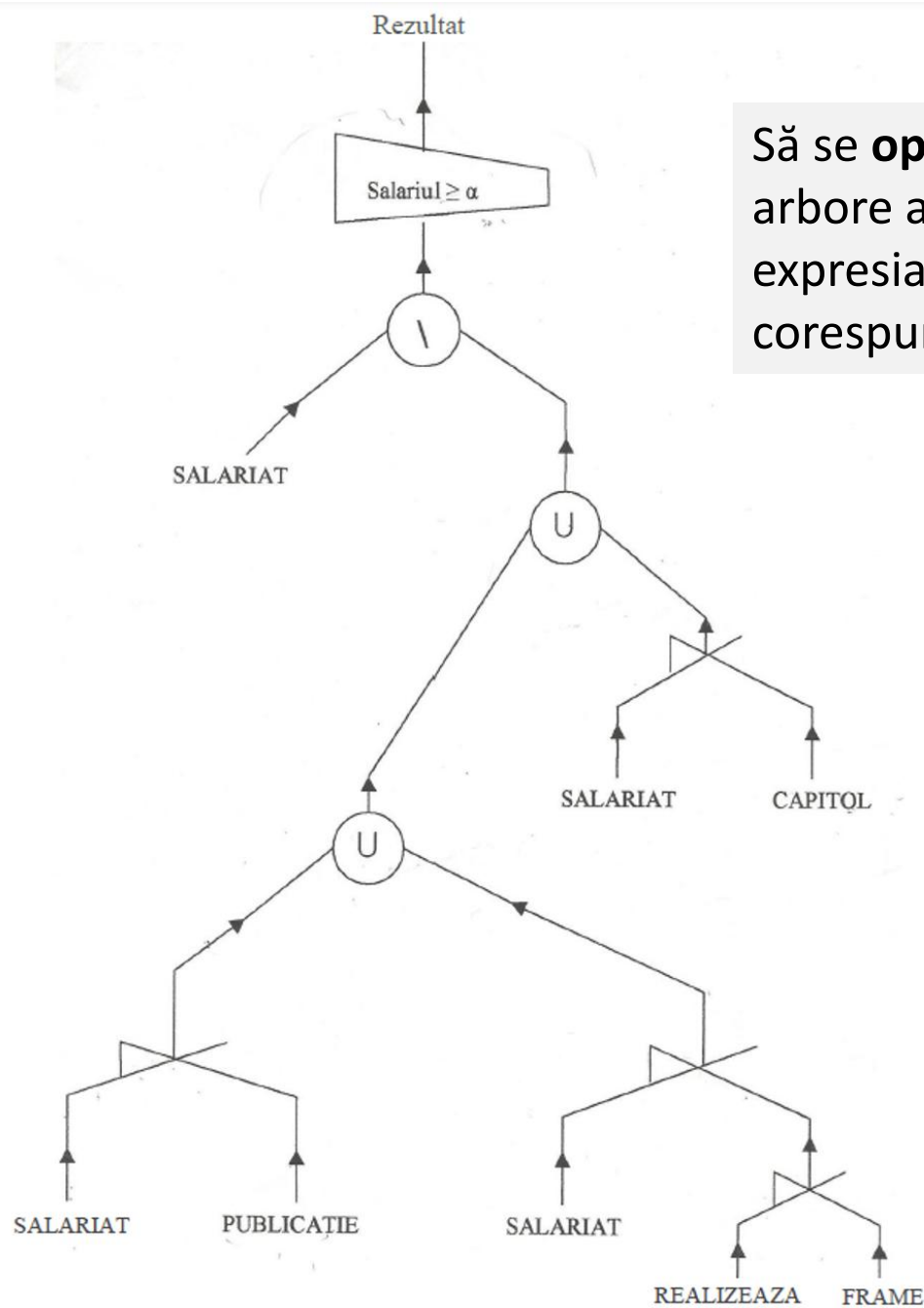
Exerciții Arbori

Rezolvările exercițiilor din cadrul acestui curs se află și pe site, în documentul numit “**Exerciții_Abori**”

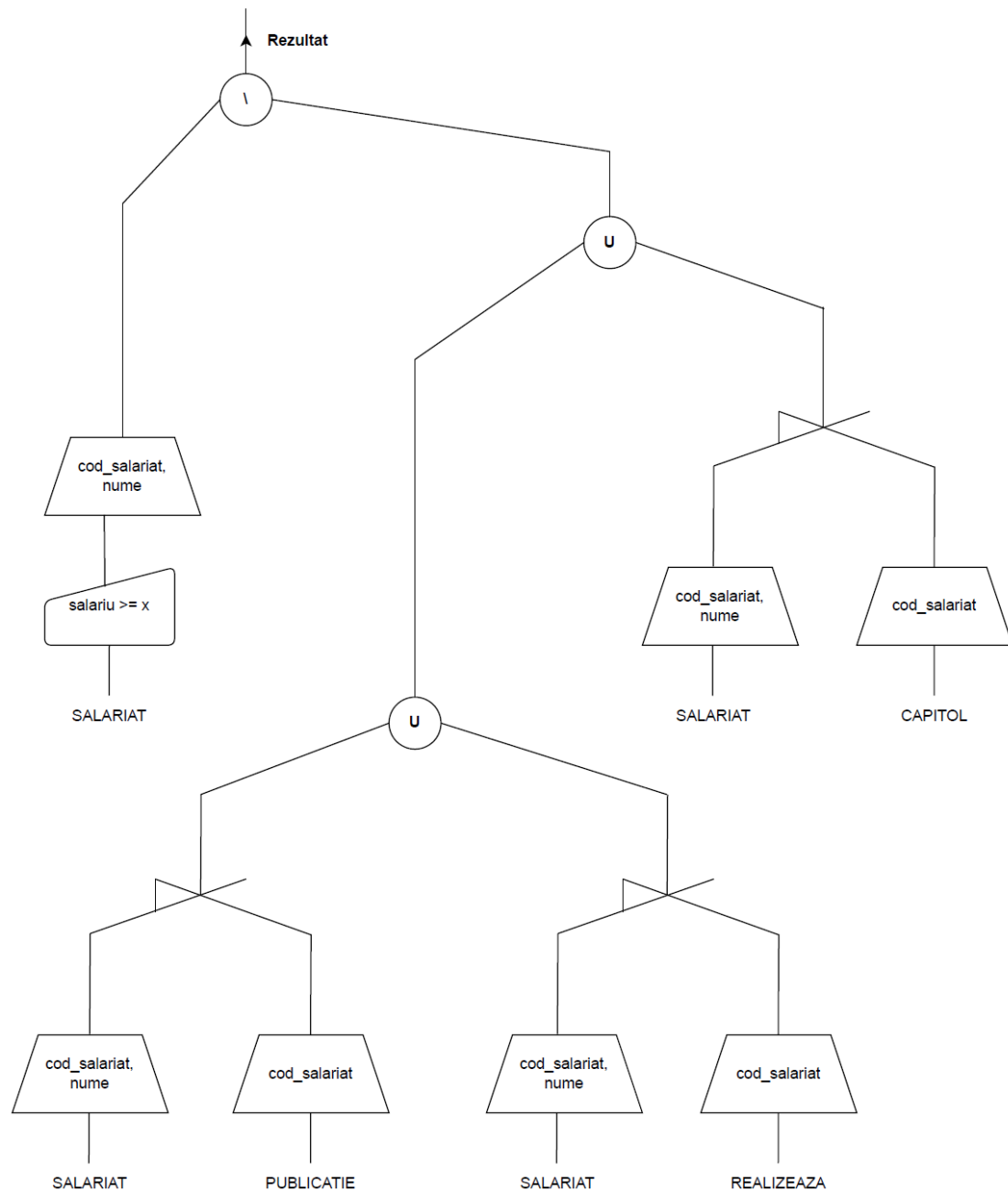
Exercițiul 1

Să se afișeze informații despre salariații care **nu** contribuie la machetarea niciunei publicații și au salariul mai mare decât o valoare data – **cerere în limbaj natural**





Să se **optimizeze** următorul
arbore algebric și să se scrie
expresia algebrică
corespunzătoare



Expresia algebrică:

R1 = PROJECT(REALIZEAZA, cod_salariat)

R2 = PROJECT(SALARIAT, cod_salariat, nume)

R3 = SEMIJOIN(R1, R2, cod_salariat)

R4 = PROJECT(PUBLICATIE, cod_salariat)

R5 = SEMIJOIN(R4, R2, cod_salariat)

R6 = UNION(R3, R5)

R7 = PROJECT(CAPITOL, cod_salariat)

R8 = SEMIJOIN(R7, R2, cod_salariat)

R9 = UNION(R8, R6)

R10 = SELECT(SALARIAT, salariu >= x)

R11 = PROJECT(R10, cod_salariat, nume)

R12 = MINUS(R11, R9)

Rezultat = PROJECT(R12, cod_salariat, nume)

Exercițiul 2

Codurile și numerele de telefon ale investitorilor, împreună cu valoarea investiției și durata de execuție a lucrărilor pentru care valoarea investiției este între două limite **I1** și **I2**, iar tipul contractantului este **'investiție'**.



Scrieți **cererea SQL**, urmată de **expresia algebrică** și **arborele algebric**.

Codurile și numerele de telefon ale investitorilor, împreună cu valoarea investiției și durata de execuție a lucrărilor pentru care valoarea investiției este între două limite **I1** și **I2**, iar tipul contractantului este '**investiție**'.



Cerere SQL?

Codurile și numerele de telefon ale investitorilor, împreună cu valoarea investiției și durata de execuție a lucrărilor pentru care valoarea investiției este între două limite **I1** și **I2**, iar tipul contractantului este **'investiție'**.



```
SELECT cod_contractant, telefon, val_investitie, durata  
FROM contractant co JOIN contract ct ON(co.cod_contractant =  
ct.cod_contractant)  
WHERE val_investitie > I1 AND val_investitie < I2 AND  
tip_contractant = 'investitie';
```

```
SELECT cod_contractant, telefon, val_investitie, durata
FROM contractant co JOIN contract ct ON(co.cod_contractant =
ct.cod_contractant)
WHERE val_investitie > l1 AND val_investitie < l2 AND
tip_contractant = 'investitie';
```

Expresia algebrică - neoptimă:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > l1)

R3 = SELECT(R2, val_investitie < l2)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon, tip_contractant)

R5 = SELECT(R4, tip_contractant = 'investitie')

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Expresia algebrică:

$R1 = \text{PROJECT}(\text{CONTRACT}, \text{cod_contractant}, \text{val_investitie}, \text{durata})$

$R2 = \text{SELECT}(R1, \text{val_investitie} > 11)$

$R3 = \text{SELECT}(R2, \text{val_investitie} < 12)$

$R4 = \text{PROJECT}(\text{CONTRACTANT}, \text{cod_contractant}, \text{telefon}, \text{tip_contractant})$

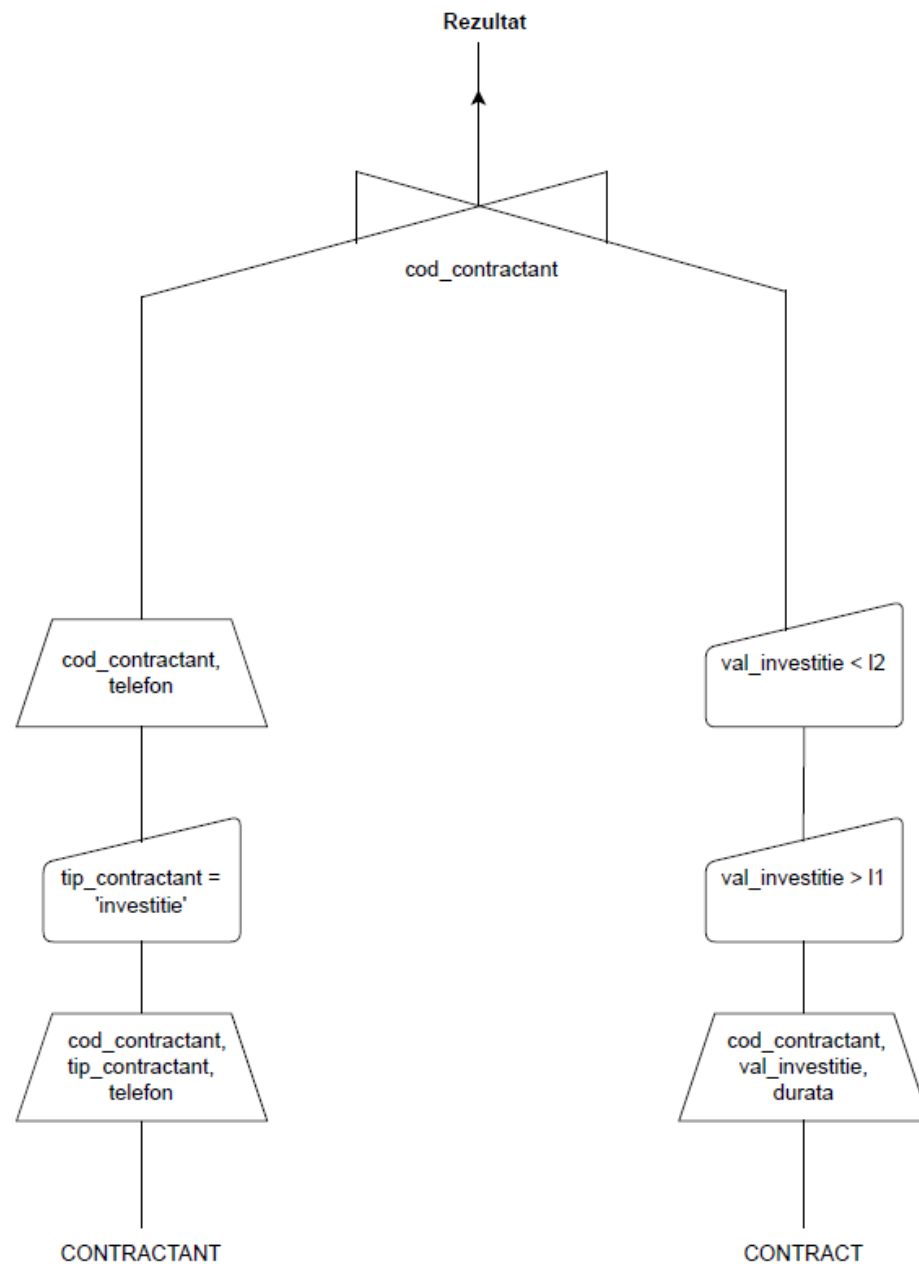
$R5 = \text{SELECT}(R4, \text{tip_contractant} = \text{'investitie'})$

$R6 = \text{PROJECT}(R5, \text{cod_contractant}, \text{telefon})$

Rezultat = $\text{JOIN}(R6, R3, \text{cod_contractant})$

Join-ul din relația **Rezultat** returnează toate coloanele pe care le au relațiile primite ca argument.

Arborele algebric - neoptimizat:



Expresia algebrică - **neoptimizată**:

R1 = PROJECT(CONTRACT, cod_contractant, val_investitie, durata)

R2 = SELECT(R1, val_investitie > 11)

R3 = SELECT(R2, val_investitie < 12)

R4 = PROJECT(CONTRACTANT, cod_contractant, telefon, tip_contractant)

R5 = SELECT(R4, tip_contractant = 'investitie')

R6 = PROJECT(R5, cod_contractant, telefon)

Rezultat = JOIN(R6, R3, cod_contractant)

Expresia algebrica - **optimizată**:

R1 = SELECT(CONTRACT, val_investitie > 11 AND val_investitie < 12)

R2 = PROJECT(R1, cod_contractant, val_investitie, durata)

R3 = SELECT(CONTRACTANT, tip_contractant = 'investitie')

R4 = PROJECT(R3, cod_contractant, telefon)

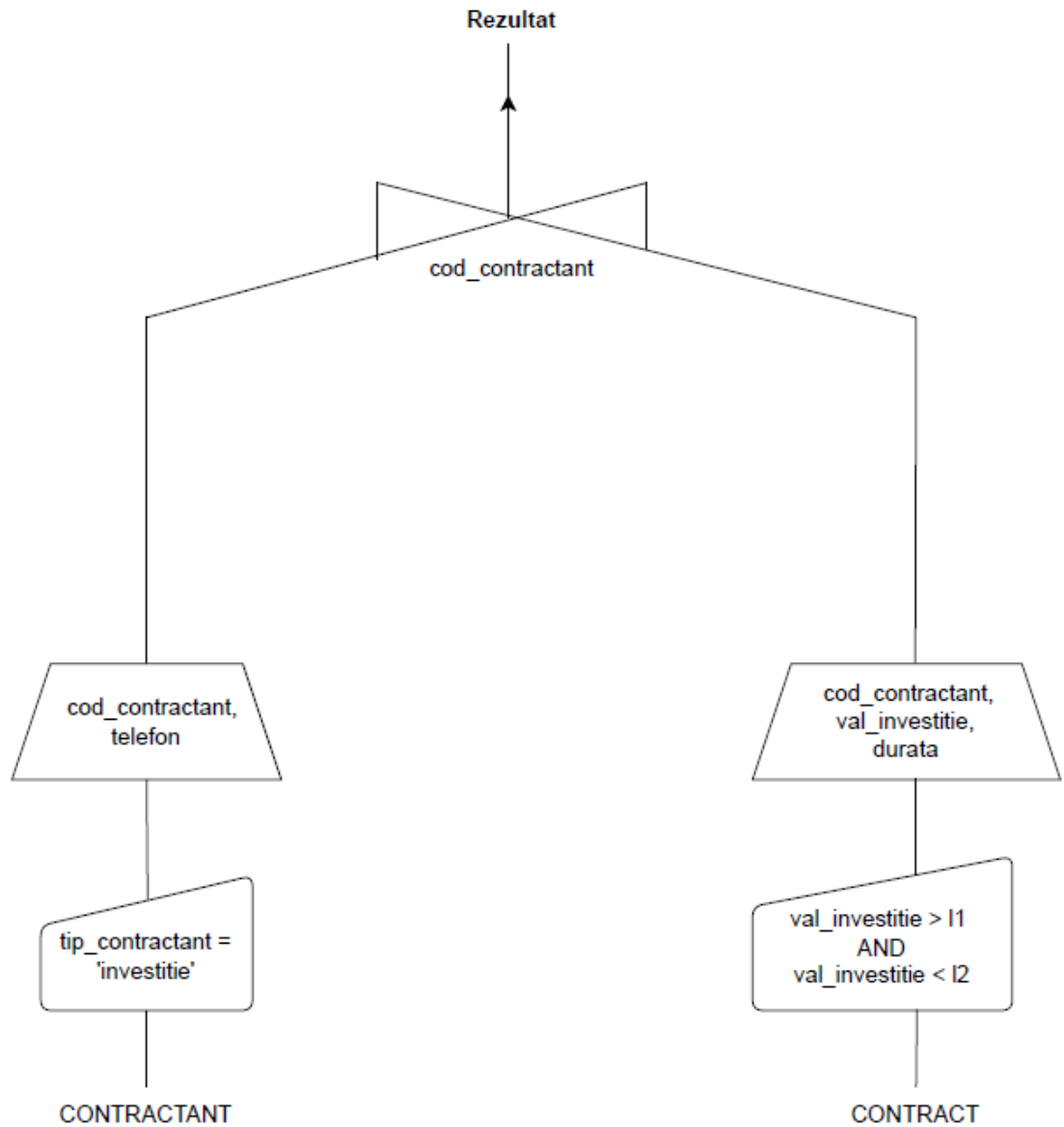
Rezultat = JOIN(R4, R2, cod_contractant) -> **rezultat optimizat**

Explicațiile detaliate și pașii optimizării se află în documentul **Exerciții_Arbori** - de pe site.

În același mod se procedează și la **EXAMENUL FINAL**.

Arborele algebric

– **optimizat:**



De ce învățăm expresii și arbori algebrici?

- SQL-ul este doar un limbaj declarativ – noi spunem "ce vrem", dar nu "cum" să se execute.
- Sistemul de gestiune a bazei de date (ex: Oracle Database, PostgreSQL, MySQL) transformă automat cererea SQL într-un arbore algebric (intern, în memorie).
- Optimizatorul bazei de date aplică reguli matematice de rescriere ca să transforme arborele într-o formă mai rapidă și mai eficientă de executat.
- Prin urmare, dacă știm să lucrăm cu expresii și arbori algebrici, știm cum "gândește" sistemul și putem scrie de la început cereri SQL mai eficiente.
- Mai eficient înseamnă **timp de răspuns mai mic, încărcare mai mică pe server, costuri mai mici.**

Cum legăm teoria de codul real?

- În SQL Developer facem **EXPLAIN PLAN** → vedem arborele de execuție (fiecare nod e un operator algebric: SELECT, JOIN, PROJECT etc.)
- Dacă cererea SQL nu este optim scrisă, arborele generat e mai lung și implică scanări inutile de date, join-uri prea multe și lente sau chiar filtrări târzii
- Dacă scriem cereri mai bine gândite (optimizate logic), arborele e mai scurt și mai eficient - filtrăm mai repede, procesăm mai puține date, facem join mai mic.

Exemplu concret pe diagrama HR - neoptimizat

Să se afișeze numele, prenumele angajaților, dar și denumirea departamentului în care aceștia lucrează. Se vor afișa doar acei angajați care au salariul mai mare decât 5000 și care lucrează în departamentele aflate în locația cu id-ul 1700.

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e JOIN departments d
      ON e.department_id = d.department_id
WHERE e.salary > 5000 AND d.location_id = 1700;
```

Expresie algebrică - neoptimizată:

```
R1 = JOIN(EMPLOYEES, DEPARTMENTS, department_id)
R2 = SELECT(R1, salary > 5000 AND location_id = 1700)
R3 = PROJECT(R2, first_name, last_name, department_name)
```

Problema:

Se realizează JOIN-ul între toți angajații și toate departamentele, după care se filtrează salariile și locațiile ⇒ se procesează prea multe date.

Exemplu concret pe diagrama HR - **optimizat**

Să se afișeze numele, prenumele angajaților, dar și denumirea departamentului în care aceștia lucrează. Se vor afișa doar acei angajați care au salariul mai mare decât 5000 și care lucrează în departamentele aflate în locația cu id-ul 1700.

```
SELECT e.first_name, e.last_name, d.department_name
FROM (SELECT * FROM employees WHERE salary > 5000) e
JOIN (SELECT * FROM departments WHERE location_id = 1700) d
ON e.department_id = d.department_id;
```

Expresie algebrică - **optimizată:**

R1 = SELECT(EMPLOYEES, salary > 5000)

R2 = SELECT(DEPARTMENTS, location_id = 1700)

R3 = JOIN(R1, R2, department_id)

R4 = PROJECT(R3, first_name, last_name, department_name)

De ce este mai optimă această variantă?

- Se realizează filtrarea **înaintea** JOINULUI ⇒ mai puține date combinate ⇒ JOIN mai mic și mai rapid.
- Se reduce încărcarea serverului.

Cum arată în planul de execuție (EXPLAIN PLAN)

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				57 6
MERGE JOIN				57 6
TABLE ACCESS	DEPARTMENTS	BY INDEX ROWID		21 2
Filter Predicates LOCATION_ID=1700				
INDEX	DEPT_ID_PK	FULL SCAN		27 1
SORT		JOIN		57 4
Access Predicates EMPLOYEES.DEPARTMENT_ID=DEPARTMENTS.DEPARTMENT_ID				
Filter Predicates EMPLOYEES.DEPARTMENT_ID=DEPARTMENTS.DEPARTMENT_ID				
TABLE ACCESS	EMPLOYEES	FULL		57 3
Filter Predicates SALARY>5000				

Ambele cereri au același plan de execuție. De ce?

De ce ambele cereri au același plan de execuție

- **Motivul principal** este că optimizatorul SQL (Oracle Database, SQL Server, PostgreSQL etc.) este foarte inteligent.
- Chiar dacă se implementează o cerere „neoptimizată”, **optimizatorul o rescrie automat** în forma cea mai bună, înainte să o execute.

În Oracle SQL Developer:

- Optimizatorul aplică automat **Predicate Pushdown** și **View Merging** – adică extrage condițiile din subcereri sau le împinge cât mai aproape de sursa de date, **chiar dacă acest lucru nu a fost făcut manual**.
- Indiferent dacă filtrările sunt scrise după join sau înainte, Oracle mută singur filtrările cât mai devreme.

Concluzii

- La nivel de învățare, este important să știm cum gândește optimizatorul.
- Chiar dacă un SGBD modern optimizează singur, în practică nu toate bazele de date sunt la fel de bune. În baze de date mai simple sau în cazuri speciale, scrierea corectă face diferența.
- În interviuri tehnice sau proiecte mari (enterprise), este important să cunoaștem ce face optimizatorul și să implementăm cereri clare și predictibile.
- Chiar dacă SQL Developer arată același plan în cazuri mai simple, în sisteme reale, cu date mari și tabele complexe, modul în care scriem SQL-ul poate influența mult performanța.
- Expresiile și arborii algebrici ne ajută să înțelegem cum “gândește” sistemul și să scriem cereri optime de la început.
- În cazuri complexe, precum bazele de date mari sau query-uri pe sute de milioane de rânduri, scrierea corectă poate duce la diferențe de 10x sau 100x în performanță.

Regulile lui Codd

Avantajele modelului relațional:

- fundamentare matematică riguroasă;
- independență fizică a datelor;
- posibilitatea filtrărilor;
- existența unor structuri de date simple;
- realizarea unei redundanțe minime;
- suplețe în comunicarea cu utilizatorul neinformatician;

Regulile lui Codd

Limite ale modelului relațional:

- rămâne totuși redundanță;
- nu lucrează cu obiecte complexe;
- nu realizează gestiunea totală a datelor distribuite;

Regulile lui Codd

- În anul 1985, E.F. Codd a publicat un set de **12 reguli** în raport cu care un sistem de gestiune a bazelor de date poate fi apreciat ca relațional
 - Niciun sistem de gestiune a bazelor de date nu respectă absolut toate regulile definite de Codd, dar acest lucru nu împiedică etichetarea acestor sisteme drept relaționale
- Nu trebuie apreciat un SGBD ca fiind relațional sau nu, ci **măsura în care acesta este relațional**, deci numărul regulilor lui Codd pe care le respectă

Regulile lui Codd

- **Regula 1 – regula reprezentării informației.** *Într-o bază de date relațională, informația este reprezentată la nivel logic sub forma unor **tabele ce poartă numele de relații**.*
- **Regula 2 – regula accesului garantat la date.** *Fiecare valoare dintr-o bază de date relațională trebuie să poată fi accesată în mod logic printr-o **combinație formată din numele relației, valoarea cheii primare și numele atributului**.*
- **Regula 3 – regula reprezentării informației necunoscute.** *Un sistem relațional trebuie să permită utilizatorului definirea unui tip de date numit „**null**” pentru reprezentarea unei informații necunoscute la momentul respectiv.*

Regulile lui Codd

- **Regula 4 – regula dicționarilor de date.** *Asupra descrierii bazelor de date (informații relative la relații, vizualizări, indecși etc.) trebuie să se poată aplica **aceleași operații** ca și asupra datelor din baza de date.*
- **Regula 5 – regula limbajului de interogare.** *Trebuie să existe cel puțin un **limbaj pentru prelucrarea bazei de date**.*
- **Regula 6 – regula de actualizare a vizualizării.** *Un SGBD trebuie să poată determina dacă o vizualizare poate fi actualizată și să stocheze rezultatul interogării într-un dicționar de tipul unui catalog de sistem.*
- **Regula 7 – regula limbajului de nivel înalt.** *Regulile de prelucrare asupra unei relații luate ca întreg sunt valabile atât pentru operațiile de regăsire a datelor, cât și asupra operațiilor de inserare, actualizare și ștergere a datelor.*

Regulile lui Codd

- **Regula 8 – regula independenței fizice a datelor:** *Programele de aplicație și activitățile utilizatorilor nu depind de modul de depunere a datelor sau de modul de acces la date.*
- **Regula 9 – regula independenței logice a datelor.** *Programele de aplicație trebuie să funcționeze normal chiar dacă structura logică a datelor se schimbă (de exemplu: adăugăm sau eliminăm un câmp dintr-un tabel).*
- **Regula 10 – regula independenței datelor privind integritatea acestora.** *Regulile de integritate trebuie să fie definite într-un sublimbaj relațional, nu în programul de aplicație.*

Regulile lui Codd

- **Regula 11 – regula independenței datelor din punct de vedere al distribuirii.** *Distribuirea datelor pe mai multe calculatoare dintr-o rețea de comunicații de date nu trebuie să afecteze programele de aplicație.*
- **Regula 12 – regula versiunii procedurale a unui SGBD.** *Orice componentă procedurală a unui SGBD trebuie să respecte aceleași restricții de integritate ca și componenta relațională.*

Regulile lui Codd

Deoarece regulile lui Codd sunt prea severe pentru a fi respectate de un SGBD operațional, s-au formulat **criterii minime de definire a unui sistem de gestiune relational**:

- Un SGBD este **minimal relațional** dacă:
 - toate datele din cadrul bazei sunt reprezentate prin **valori în tabele**;
 - nu există pointeri observabili de către utilizator;
 - sistemul suportă operatorii relaționali de **proiecție, selecție și compunere naturală**, fără limitări impuse din considerente interne;
- Un SGBD este **complet relațional** dacă este minimal relațional și satisface în plus condițiile:
 - sistemul suportă **restricțiile de integritate de bază** (unicitatea cheii primare, constrângerile referențiale, integritatea entității);
 - sistemul suportă **toate operațiile de bază ale algebrei relaționale**;

Normalizarea Relațiilor

- În procesul modelării unei baze de date relaționale, o etapă importantă o reprezintă **normalizarea relațiilor conceptuale**, adică obținerea de relații „moleculare” fără a pierde nimic din informație, pentru a elimina:
 - **redundanța;**
 - **anomaliile reactualizării informațiilor.**

Normalizarea Relațiilor

- Tehnica normalizării permite:
 - obținerea unei **scheme conceptuale rafinate** printr-un proces de **ameliorare progresivă** a unei scheme conceptuale inițiale a bazei de date relaționale;
- După fiecare etapă de ameliorare, relațiile bazei de date ating un anumit **grad de perfecțiune**, deci se află într-o anumită **formă normală**
 - Trecerea unei relații dintr-o formă normală în alta, presupune **eliminarea unui anumit tip de dependență** nedorită, care este transformată în dependență admisibilă, adică dependență care nu provoacă anomalii;

Normalizarea Relațiilor

Procesul de ameliorare a schemei conceptuale trebuie:

- să garanteze **conservarea datelor**, adică în schema conceptuală finală trebuie să figureze toate datele din cadrul schemei inițiale;
- să garanteze **conservarea dependențelor** dintre date, adică în schema finală fiecare dependență trebuie să aibă determinantul și determinatul în schema aceleiași relații;
- să reprezinte o **descompunere minimală** a relațiilor inițiale, adică niciuna dintre relațiile care compun schema finală nu trebuie să fie conținută într-o altă relație din această schema;

Normalizarea Relațiilor

Există două metode pentru a modela **baze de date relaționale fără anomalii sau pierderi de informație**:

- **Schema descompunerii** pleacă de la o schemă relațională universală ce conține toate attributele BD.
 - Schema se descompune prin **proiecții succesive** în subrelații;
 - Descompunerea se oprește când continuarea ei ar duce la pierderi de informație;
 - Algoritmii de descompunere se bazează, în general, pe descrierea formală a dependenței dintre attribute;

Normalizarea Relațiilor

- **Schema sintezei** pleacă de la o mulțime de attribute independente.
 - Utilizând proprietăți de semantică și legături între attribute se pot compune noi relații, astfel încât, acestea să nu sufere de anumite anomalii;
 - Algoritmii se bazează, în general, pe teoria grafurilor pentru a reprezenta legăturile dintre attribute;

Dependențe funcționale

- O **relație universală** este o relație ce grupează toate atributele care modelează sistemul real cercetat
- Fie E , **mulțimea dependențelor** considerate de proiectantul bazei de date pentru o schemă relațională sau pentru o relație universală.
 - Plecând de la o mulțime de proprietăți formale ale dependențelor, proprietăți considerate drept reguli de deducție (**axiome**), poate fi obținută **mulțimea maximală de dependențe asociate lui E** . Această mulțime definește închiderea lui E .

Dependențe funcționale

- Fie E mulțimea dependențelor unei relații și $p_1, p_2, \dots, p_r, r \geq 1$, proprietăți formale ale acestor dependențe.
 - Dacă există o mulțime E' , astfel încât orice dependență a mulțimii E este derivabilă din E' prin aplicarea proprietăților $p_1, p_2, \dots, p_r$, atunci mulțimea E' definește **acoperirea** lui E pentru proprietățile $p_1, p_2, \dots, p_r$.
- E' este **o acoperire minimală** pentru E , dacă nu există nicio submulțime proprie, nevidă a lui E' care să fie o acoperire pentru E . **Evident, E și E' au închideri identice, deci dispun de același potențial informațional!**

Dependențe funcționale

- Fie $R(A_1, A_2, \dots, A_n)$ o schemă relațională și fie X, Y submulțimi de attribute ale lui R .
 - X **determină funcțional** Y sau Y depinde funcțional (FD) de X , dacă pentru orice relație r (valoare curentă a lui R) **nu există două tupluri care să aibă aceleași valori pentru attributele lui X și să aibă valori diferite pentru cel puțin un atribut din Y** . Cu alte cuvinte, o valoare a lui X , determină unic o valoare a lui Y .
- Notăție: $X \rightarrow Y$; X este numit **determinant**, iar Y este numit **determinat** (sau dependent).
 - Dependența funcțională $X \rightarrow Y$ este **trivială** dacă $Y \subseteq X$.

Dependențe funcționale

În momentul proiectării și modelării unei baze de date relaționale, pot să apară anomalii. Procesul de optimizare a relațiilor conceptuale poartă numele de **normalizare**. Pentru a depista eventualele redundanțe se utilizează **dependențele funcționale**.

Fie $R(A_1, A_2, \dots, A_n)$ o schemă relațională.

De exemplu:

EMPLOYEES(employee_id, first_name, salary, job_id, job_title)

EMPLOYEES

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Tipuri de anomalii:

- 1) **Redundanța** – stocarea repetată a unor valori (în exemplul anterior IT_PROG și PROGRAMATOR).
- 2) **Anomalii de actualizare** – modificarea valorii redundante de la 1) poate duce la modificarea valorii doar pentru unele înregistrări, și nu pentru toate.
- 3) **Anomalii de inserare** – nu se poate insera un job (job_id, job_title) decât dacă se inserează, în baza de date, și un angajat asociat.
- 4) **Anomalii de ștergere** – dacă se șterge un angajat, se pierde automat și datele despre job.

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Apar aceste anomalii deoarece în relația **EMPLOYEES** sunt attribute care aparțin unor mulțimi diferite X și Y și deci unor relații diferite.

X → Y - **X determină funcțional pe Y sau Y depinde funcțional de X** dacă pentru orice relație **nu există** două tupluri care să aibă aceleași valori pentru attributele lui X și să aibă valori diferite pentru cel puțin un atribut din Y. Cu alte cuvinte, o valoare a lui X **determină unic** o valoare a lui Y

De exemplu:

100 KING 24000 **IT_PROG PROGRAMATOR**

101 JOHN 9500 **MGR MANAGER**

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

De exemplu:

100 KING 24000 **IT_PROG PROGRAMATOR**

101 JOHN 9500 **MGR MANAGER**

IT_PROG determină funcțional **PROGRAMATOR**

Deci, o valoare a lui X determină unic o valoare a lui Y. Astfel, se obțin următoarele submulțimi de attribute ale relației EMPLOYEES:

{job_id} -> {job_title} – job_id determină funcțional pe job_title

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Se observă următoarele submulțimi de attribute:

$X' = \{\text{employee_id}, \text{first_name}, \text{salary}, \text{job_id}\}$

$Y' = \{\text{job_id}, \text{job_title}\}$

Din submulțimile X' și Y' de attribute, se obțin următoarele scheme relaționale:

EMPLOYEES(employee_id, first_name, salary, job_id)

JOBS(job_id, job_title)

Dependențe funcționale

emp_id	first_name	salary	job_id	job_title
100	KING	24000	IT_PROG	PROGRAMATOR
101	JOHN	9500	MGR	MANAGER
102	HUNOLD	17000	IT_PROG	PROGRAMATOR

Se observă că informația din relația **EMPLOYEES** trebuie, de fapt, stocată în două relații, astfel:

EMPLOYEES

emp_id	first_name	salary	Job_id
100	KING	24000	IT_PROG
101	JOHN	9500	MGR
102	HUNOLD	17000	IT_PROG

JOBS

job_id	Job_title
IT_PROG	PROGRAMATOR
MGR	MANAGER

Dependențe funcționale

Acest procedeu se numește **descompunerea relațiilor**.

Descompunerile se realizează pe baza formelor normale pe care le vom studia în cursul următor, iar definirea formelor normale se bazează pe noțiunea de ***dependență funcțională***.

Dependențe funcționale

EMPLOYEES

emp_id	first_name	salary
100	KING	24000
101	JOHN	9500
102	HUNOLD	17000

JOBS

job_id	Job_title
IT_PROG	PROGRAMATOR
MGR	MANAGER

În exemplul anterior, există următoarele dependențe funcționale:

employee_id -> first_name, salary (se citește: employee_id **determină funcțional** first_name și salary sau first_name și salary **depind funcțional** de employee_id)

job_id -> job_title

Dependențe funcționale

Pornind de la aceste dependențe, putem afirma că:

- Dacă doi angajați ar avea același `employee_id` – înseamnă că este același angajat;
- Dacă doi angajați au același `job_id` – atunci rezultă că au, implicit, și același `job_title`;

Dependențe funcționale

- Comparând toate submulțimile de attribute ale unei relații și determinând legăturile dintre ele, se pot obține toate dependențele funcționale pe care o relație le satisface.
 - Această abordare **nu** este eficientă, consumând mult timp;
- Există posibilitatea ca, știind **anumite dependențe** funcționale și utilizând **reguli de deducție**, să fie obținute **toate** dependențele funcționale.

Dependențe funcționale

Fie X, Y, Z, W mulțimi de attribute ale unei scheme relaționale R și fie următoarele axiome:

- **Ax1 – reflexivitate.** $X \rightarrow X$. Mai general, dacă $Y \subseteq X$, atunci $X \rightarrow Y$.
- **Ax2 – creșterea determinantului (augmentare).** Pot fi considerate următoarele formulări echivalente pentru această axiomă.
 - Dacă $X \rightarrow Y$ și $X \subseteq Z$, atunci $Z \rightarrow Y$. ($\subseteq \rightarrow$ fiecare element din X este și element în Z ; X este inclus în Z)
 - Dacă $X \rightarrow Y$ și $W \subseteq Z$, atunci $X \cup Z \rightarrow Y \cup W$.
 - Dacă $X \rightarrow Y$ atunci $X \cup Z \rightarrow Y \cup Z$.
 - Dacă $X \rightarrow Y$ atunci $X \cup Z \rightarrow Y$.

În urma reuniunii se obțin dependențe funcționale valide

- **Ax3 – tranzitivitate.** Dacă $X \rightarrow Y$ și $Y \rightarrow Z$, atunci $X \rightarrow Z$.

Dependențe funcționale

- O mulțime de axiome este **completă** dacă și numai dacă plecând de la o mulțime de dependențe E se pot obține toate dependențele închiderii lui E , utilizând axiomele mulțimii. Cu alte cuvinte, se pot obține TOATE dependențele corecte folosind axiomele.
- O mulțime de axiome este **închisă** dacă și numai dacă plecând de la o mulțime de dependențe E , nu poate fi dedusă cu ajutorul axiomelor o dependență care nu aparține închiderii lui E . Cu alte cuvinte, nu se pot obține dependențe GREȘITE aplicând axiomele.
- **Ullman** a demonstrat că axiomele Ax1 – Ax3, numite **axiomele lui Armstrong**, reprezintă o mulțime închisă și completă de axiome.

Dependențe funcționale

- Nu toate dependențele funcționale sunt folositoare pentru modelarea relațională.
- O dependență funcțională $X \rightarrow Y$ se numește **dependență funcțională totală** (FT), dacă și numai dacă nu există nicio submulțime proprie $X' \subset X$, astfel încât $X' \rightarrow Y$.
 - Dacă există o submulțime proprie $X' \subset X$, astfel încât $X' \rightarrow Y$, atunci dependența funcțională $X \rightarrow Y$ este parțială. În axioma Ax2, dependența $Z \rightarrow Y$ este o dependență funcțională parțială;

Dependențe funcționale

- În cazul dependenței funcționale totale, axiomele lui Armstrong se reduc la o axiomă unică și anume **pseudo-tranzitivitatea**:
 - dacă $X \rightarrow Y$ și $W \cup Y \rightarrow Z$, atunci $W \cup X \rightarrow Z$
- Această axiomă este o regulă de deducție completă pentru total dependențe:
 - pseudo-tranzitivitatea implică tranzitivitatea (dacă $W = \emptyset$);
 - reflexivitatea nu poate fi utilizată pentru a obține dependențe totale;

Dependențe funcționale

- Dacă F este o mulțime de dependențe funcționale totale, atunci **închiderea pseudo-tranzitivă** F^+ a acestei mulțimi este reuniunea mulțimilor dependențelor funcționale totale care pot fi obținute din F folosind axioma de pseudo-tranzitivitate. Multimea tuturor dependentelor functionale care se pot deduce din F se numeste inchiderea multimii de dependente F , notata cu F^+
- Două mulțimi de dependențe funcționale totale sunt **echivalente** dacă au **închideri pseudo-tranzitive identice**

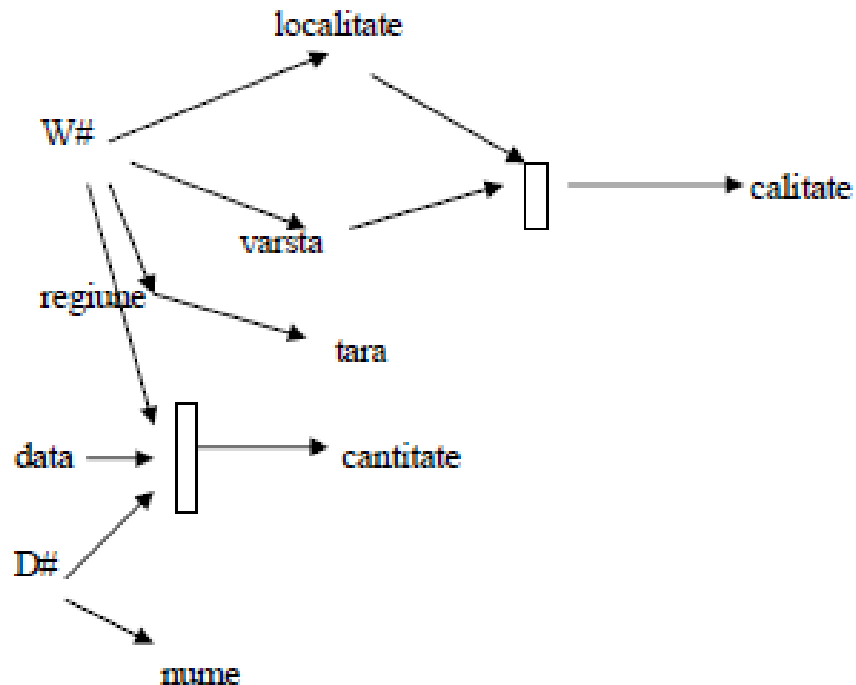
Dependențe funcționale

- Dependențele funcționale între atributele bazei pot fi reprezentate grafic
 - Fie $A = \{A_1, A_2, \dots, A_n\}$ o mulțime de atribute și fie o mulțime de dependențe funcționale $\{X_i \rightarrow A_j\}$, unde X_i este o submulțime a lui A .
- **Graful dependențelor funcționale** este un **graf direcționat bipartit**, definit astfel:
 1. pentru fiecare atribut A_j există un singur **nod** având eticheta A_j ;
 2. pentru fiecare dependență funcțională de forma $A_i \rightarrow A_j$, există un **arc** de la A_i la A_j ;
 3. pentru fiecare dependență funcțională de forma $X_i \rightarrow A_j$, unde mulțimea X_i este definită de $X_i = \{A_{i_1}, \dots, A_{i_p}\}$ cu $p > 1$, există un **nod auxiliar** etichetat prin X_i și o **mulțime de arce** plecând de la $A_{i_1}, \dots, A_{i_p}$ pentru a obține pe X_i și **printr-un arc adițional** de la X_i la A_j . Nodurile X_i se reprezintă prin dreptunghiuri.

Dependențe funcționale

Exemple:

1. Graful dependențelor funcționale pentru schema relațională **CONSUMATOR_DE_VIN**(W#, localitate, varsta, calitate, regiune, tara, D#, nume, data, cantitate).



Dependențe funcționale

2. Graful dependențelor funcționale pentru schema relațională **OBIECTIV_INVESTITIE**. Dependentele sunt deduse din regulile impuse de beneficiar.

